

Base de Datos (75.15 / 95.05 / TA044)

Segundo Parcial Promocional - 4 de diciembre de 2024 - 2024241

Costos		Costos		NoSQL		Padrón: Apellido: Nombre: Hojas entregadas:
NoSQL		Conc.		Recup.		
Nota: ☐ Aprobado ☐ Insuficiente						

Criterio de aprobación: El examen está compuesto por 6 ítems, cada uno de los cuales se corrige como B/B-/Reg/Reg-/M. El examen se aprueba con nota mayor o igual a 4 (cuatro) y la condición de aprobación es desarrollar un ítem bien (B/B-) en los 3 grupos de ejercicios (costos, NoSQL, concurrencia/recuperación). Adicionalmente, no deberá haber más de dos ítems mal o no desarrollados

1. (*Costos*) La famosa banda de rock “*Eruca Sativa*” está buscando lugares donde presentar su nuevo hit “*Lío*”, con el objetivo de estar en las últimas provincias del país en las que no han tocado aún. Para ello utilizan una base de datos relacional con el siguiente esquema:

- **Antros** (id_antro, nombre, direccion, ciudad, provincia, capacidad)
- **Disponibilidad** (id_antro, fecha)

Se busca disponibilidad para 50 mil personas en 5 días de enero, con la siguiente consulta:

```
SELECT a.id_antro, nombre
FROM antros a INNER JOIN disponibilidad d USING(id_antro)
WHERE (a.provincia = 'Catamarca' OR a.provincia = 'La Rioja')
AND a.capacidad >= 50,000
AND (d.fecha BETWEEN '2025-01-06' AND '2025-01-10')
```

Estime el **costo** de realizar esta consulta y **la cantidad de filas** que serán devueltas, contando la siguiente información de catálogo, sabiendo que se dispone de 100 bloques de memoria disponibles para la operación, que los índices indicados no son de clustering y que un 10% de los antros tienen capacidad para al menos 50 mil personas:

ANTROS	DISPONIBILIDAD
n(antros) = 10.000	n(disponibilidad) = 100.000
B(antros) = 2.000	B(disponibilidad) = 5.000
V(provincia, antros) = 20	V(id_antro, disponibilidad) = 10.000
	V(fecha, disponibilidad) = 100
H(I(provincia, antro)) = 2	H(I(id_antro, disponibilidad)) = 4
	H(I(fecha, disponibilidad)) = 3

2. (*Costos*) La plataforma de juegos “Vapor” quiere vincular a sus usuarios con los juegos de sus géneros favoritos. Para ello, quiere hacer un join entre las siguientes dos tablas por igualdad del atributo `id_genero`, utilizando el método de hash grace.

- GenerosFavoritos (`id_usuario`, `id_genero`)
- Juegos (`id_juego`, `nombre`, `id_genero`)

Estime el **costo** de realizar esta junta e indique la **cantidad mínima de memoria** que se necesita para hacerla.

GENEROSFAVORITOS	JUEGOS
$n(\text{GenerosFavoritos}) = 100.000$	$n(\text{Juegos}) = 50.000$
$B(\text{GenerosFavoritos}) = 1.000$	$B(\text{Juegos}) = 5.000$
$V(\text{id_genero}, \text{GenerosFavoritos}) = 90$	$V(\text{id_genero}, \text{Juegos}) = 100$

3. (*NoSQL*) Preocupado por la baja concurrencia a sus clases, un conocido profesor hace lo que cualquiera haría: vuelca los datos que tiene en una base Neo4j para poder consultarlos y tomar decisiones.

Crea nodos para los alumnos y las clases, y vínculos indicando qué alumno asistió a qué clase y de qué modo (las clases presenciales pueden ser asistidas en modo virtual o presencial, las virtuales únicamente en modo virtual).

```
CREATE (a1: Alumno { padron: 12345, nombre: "Adalberto Fulco", nota: 8 } );
CREATE (c1: Clase { fecha: 2024-11-05, tema:"NOSQL", tipo: "PRESENCIAL" } );
```

```
MATCH (a1: Alumno{ padron: 12345}), (c1: Clase{ fecha: 2024-11-05})
CREATE (a1)-[:ASISTIO {tipo:"VIRTUAL"} ]->(c1);
```

Para verificar que asistir a las clases ayuda a aprobar la materia, el conocido profesor quiere encontrar aquellos alumnos (padrón y nombre) que han asistido presencialmente a todas las clases presenciales pero que no han aprobado la materia (su nota menor que 4). Se pide hacer una consulta en lenguaje Cypher que devuelva esos datos.

4. (NoSQL) El famoso restaurante “*Il Famoso Ristorantino*” hace una encuesta anual en la que sus clientes votan los mejores platos en distintas categorías. Por comodidad, registran los resultados históricos de la encuesta en una base de datos MongoDB con la siguiente estructura de documento:

```
{
  año: 2023,
  fecha_ceremonia: 2023-12-09,
  presentador: "Ananá Ferreyra",
  ganadores: [
    { tipo: "Mejor", plato: "Carré de Cerdo a la Beiranesca" },
    { tipo: "Economico", plato: "Focaccia de Atún" },
    , (...)
  ]
}
```

Haga una consulta que devuelva el nombre de los platos que, **desde el año 2010 en adelante**, hayan ganado algún premio en **al menos 3 años** (no tiene por qué ser el mismo tipo de premio). **Importante:** Considere que un plato puede haber ganado más de un premio el mismo año.

5. (Concurrencia y Transacciones) Dado el siguiente solapamiento de transacciones:

$b_{T1}; R_{T1}(X); b_{T2}; R_{T2}(X); W_{T2}(Y); b_{T3}; W_{T3}(Z); R_{T1}(Z); W_{T3}(X); R_{T1}(Y)$

- a) Indique, con una justificación mínima, si el solapamiento es serializable. En caso de serlo, indique un orden de transacciones equivalente.
- b) Agregue, luego de las operaciones, los commits de todas las transacciones para que el solapamiento sea recuperable. En caso de no ser posible, justificarlo.
- c) Indique si el solapamiento con los commits agregados permite rollbacks en cascada.

6. (Recuperación) Un SGBD implementa el algoritmo de recuperación REDO con checkpoint activo. Luego de una falla, el sistema encuentra el siguiente archivo de log (a la derecha):

Explique **cómo se llevará a cabo** el procedimiento de recuperación, indicando **hasta qué punto del archivo de log se deberá retroceder**, y **qué cambios** deberán ser realizados en disco y en el archivo de log.

01 (BEGIN, T1);
02 (WRITE T1, A, 10);
03 (BEGIN, T2);
04 (WRITE T1, B, 20);
05 (COMMIT, T1);
06 (WRITE T2, B, 30);
07 (BEGIN CKPT, T2);
08 (BEGIN, T3);
09 (WRITE T3, C, 40);
10 (COMMIT, T2);
11 (BEGIN, T4);
12 (WRITE T4, D, 50);
13 (END CKPT);
14 (COMMIT, T4);